

TEST REPORT

LLM Security Lab

A02: Mini Open Source Cybersecurity Tool
ICCS423 Computer Security

Result	Tool version	Code commit tested	Test date
7 / 7 test cases passed	0.1.0	97240a1	15 July 2026 (Asia/Bangkok)

This report checks one practical question: can a fresh user run the tool, get useful results, and receive clear errors when the input is wrong? For the tested version, yes.

All evidence below came from real commands. No live model provider was contacted, and no real secret was used.

1. Test Environment

Item	Tested value
Operating system	macOS / Darwin 25.3.0, arm64
Python	Python 3.12.7
Project version	llmsec-lab 0.1.0
Commit	97240a1
Runtime dependencies	None
Network used	No network for offline demo; localhost only inside adapter unit tests

2. What Was Tested

The checks cover normal use, invalid configuration, output files, package installation, and adapter behavior. The full unit suite also checks malformed JSON, duplicate test IDs, unsupported signals, invalid regular expressions, timeouts, oversized responses, secret redaction, HTTPS enforcement, and cross-origin redirects.

The built-in vulnerable and protected targets are deterministic teaching fixtures. They test the scanner pipeline; they are not real language models.

3. Test Cases

Test	Command / input	Expected	Actual	Status
T1 offline comparison	<code>python3 -m llmsec compare --no-color</code>	Protected fixture scores better. Vulnerable has 18 fails; protected has 24 passes.	18 fails and 6 passes at 85.00% risk; 24 passes at 0.00% risk. Exit 0.	PASS
T2 JSON report	<code>python3 -m llmsec scan --target demo-protected --format json --output /tmp/llmsec-a02-protected.json</code>	Valid schema v1 JSON with 24 passes and no errors.	JSON parsed with <code>python3 -m json.tool</code> . Counts: 24 pass, 0 fail, 0 error.	PASS
T3 bad target config	<code>python3 -m llmsec scan --target openai-compatible</code>	Clear message for missing URL/model and exit code 2.	Printed: <code>--base-url</code> and <code>--model</code> are required for <code>openai-compatible</code> . Exit 2.	PASS
T4 missing Gemini key	<code>env -u GEMINI_API_KEY python3 -m llmsec scan --target gemini --model gemini-3.5-flash</code>	Clear environment-key error and exit code 2.	Printed: <code>GEMINI_API_KEY</code> is required. Exit 2.	PASS
T5 automated suite	<code>python3 -m unittest discover -s tests -v</code>	All core, runner, CLI, reporting, Gemini, and local HTTP adapter tests pass.	Ran 30 tests in 2.682s. Result: OK.	PASS
T6 fresh install	<code>pip wheel ...; pip install --target /tmp/...; run compare from /tmp</code>	Wheel builds and installed package works outside the source tree.	Built and installed <code>llmsec-lab 0.1.0</code> . Installed comparison returned <code>protected_better: true</code> .	PASS
T7 suite listing	<code>python3 -m llmsec list-tests</code>	List all 24 safe checks across six categories.	Printed 24 checks: <code>pi</code> , <code>de</code> , <code>hc</code> , <code>jb</code> , <code>pb</code> , and <code>ta</code> groups.	PASS

4. Terminal Captures

These are short terminal captures from the commands in the test table. Long finding details were left out so the important result stays easy to read.

```
Capture A - offline comparison
$ python3 -m llmsec compare --no-color
LLM Security Lab Demo Comparison
Comparison: PASS
Target: demo-vulnerable
Summary: pass: 6 fail: 18 inconclusive: 0 error: 0
Risk: 85.00% (CRITICAL)
Coverage: 100.00%
Target: demo-protected
Summary: pass: 24 fail: 0 inconclusive: 0 error: 0
Risk: 0.00% (LOW)
Coverage: 100.00%

Capture B - full automated test suite
$ python3 -m unittest discover -s tests -v
...
test_gemini_target_uses_official_openai_compatible_endpoint_and_env_key ... ok
test_rejects_api_key_with_plain_http_base_url ... ok
test_rejects_cross_origin_redirects ... ok
test_timeout_must_be_positive_and_finite ... ok
-----
Ran 30 tests in 2.682s
OK
```

4. Terminal Captures (continued)

```

Capture C - useful configuration errors
$ python3 -m llmsec scan --target openai-compatible
--base-url and --model are required for openai-compatible
$ echo $?
2
$ env -u GEMINI_API_KEY python3 -m llmsec scan \
  --target gemini --model gemini-3.5-flash
GEMINI_API_KEY is required
$ echo $?
2

Capture D - wheel build and installed-package run
$ python3 -m pip wheel --no-deps --no-build-isolation . -w /tmp/llmsec-a02-dist
Successfully built llmsec-lab
$ python3 -m pip install --no-deps --target /tmp/llmsec-a02-installed \
  /tmp/llmsec-a02-dist/llmsec_lab-0.1.0-py3-none-any.whl
Successfully installed llmsec_lab-0.1.0
$ cd /tmp && PYTHONPATH=/tmp/llmsec-a02-installed \
  python3 -m llmsec compare --format json --output installed-report.json
exit code: 0
protected_better: true

```

5. Known Limitations

The 24 checks use deterministic text markers and a custom rule-based evaluator. A pass is useful evidence, but it is not proof that a chatbot is secure. Real model behavior can change by provider, model version, system prompt, and randomness. This report did not send requests to a paid or hosted model because no credential was needed for assignment testing. The network adapter was tested against controlled localhost servers instead.

6. Safety Note

The test data uses fake canaries and harmless markers. The tool should only be used on systems the user owns or has permission to assess. API keys must stay in environment variables and must never be included in reports, screenshots, commands, or committed files.

7. Conclusion

The tested version meets the assignment flow: it runs offline, shows a clear comparison, creates readable text and JSON reports, rejects bad configuration, installs from a wheel, and passes all 30 automated tests. The result is ready for a fresh-clone demonstration.